GCCX • ROUND 1 TECHNICAL PRESENTATION

Candidate Triage Tool

A simple recruiter workspace for searching, organizing, and reviewing candidate profiles with AI-assisted summaries.

Flask REST API | **SQLite** | **HTML / CSS / JS** | **OpenAI-compatible API**

AI supports recruiter judgment; it does not make hiring decisions.

Why build this tool?

The aim is to reduce manual triage effort while keeping candidate evaluation evidence-based.

The problem

Candidate details are spread across profiles, notes, and skills. Searching and tracking decisions manually takes time.

The solution

One workspace to search, filter, review, compare, shortlist, tag, and request concise AI summaries.

The guardrail

Summaries should reflect available profile evidence. Recruiters verify the source details and make final decisions.

Success looks like: faster candidate discovery • saved recruiter decisions • transparent AI limitations

A recruiter journey in five steps

Keep the live demo focused on one realistic task.

1. **1. Search** — Find candidates by name, role, skills, or notes.
2. **2. Filter** — Narrow results by relevant criteria.
3. **3. Review** — Open a profile and inspect skills and experience.
4. **4. Organize** — Shortlist and add tags; verify changes persist.
5. **5. Summarize** — Request an evidence-grounded AI summary.

Three layers, clear responsibilities

A small full-stack application that is straightforward to explain and test.

Frontend

HTML5, CSS3, Vanilla JavaScript

Shows candidate cards, search/filter controls, tags, shortlist state, and AI summaries.

Backend

Python + Flask REST API

Validates requests, queries records, updates recruiter state, and calls the LLM when requested.

Data + AI

SQLite + LLM API

SQLite stores candidate and recruiter data. The model generates a concise summary from provided context.

Example flow: Frontend request → Flask endpoint → SQLite or LLM service → JSON response → updated UI

What matters most in the implementation

Simple engineering choices improve reliability, transparency, and trust.

Persistence: use safe database initialization so restarting the app does not duplicate seed data or erase recruiter edits.

Search performance: use appropriate database indexes; for repeated skill lookups, an inverted index can map each skill to candidate IDs.

AI integrity: do not return fake summaries. If the API key is missing or the service fails, show a clear error state.

Privacy: keep API keys in environment variables and exclude secrets from version control.

Human oversight: show source evidence alongside AI summaries and avoid autonomous accept/reject decisions.

Rank candidates and review AI-generated code

The interviewer wants to see the reasoning, not just the final code.

Task 2a • Skill overlap

1. Compare candidate skills with required skills.
2. Count the overlap to produce a numeric score.
3. Sort highest score first.
4. Define tie-breaking, such as name then ID.
5. Test full, partial, and zero matches.

Task 2b • Debugging

Check for common mistakes:
- Loop goes past the last array item.
- Variable is not declared.
- Score is stored as an array, not a number.
- Sort direction is reversed or arithmetic returns NaN.

Fix, then test edge cases.

Explain: purpose → inputs → logic → output → tests.

Use AI to accelerate; verify every important claim

The collaboration log should record what actually happened during development.

Where AI helped

Initial code scaffolding, test ideas, debugging suggestions, documentation, or synthetic test-data generation—only include tasks you actually used.

Where you corrected it

Document one real AI suggestion that was wrong or incomplete, what you changed, and why the correction was necessary.

How you verified

Run tests, check expected outputs, try edge cases, inspect database persistence, and test real API errors.

Include 2–3 actual prompts verbatim in the AI log. Do not invent prompts, test results, or performance claims.

What should happen before a real client rollout?

For a larger candidate volume, prioritize safety and reliability before adding complexity.

Build first

- Authentication and role-based access
- Audit trail for shortlist decisions
- Background processing for slow AI calls
- Monitoring and reliable error handling

Defer for now

- Autonomous hiring or auto-rejection
- Complex multi-client billing
- Vector search without a clear need
- Automated candidate outreach

Main risk

Automation bias: recruiters may over-trust fluent AI summaries.

Mitigate by keeping source notes visible, requiring human review, and auditing summary quality.

Measure success with shortlist time, data integrity, and human agreement with extracted evidence.

LIVE WALKTHROUGH

One clear demo. One clear takeaway.

Search by skill or role and apply a filter.

Open a profile and inspect the evidence.

Shortlist the candidate, add a tag, and refresh to verify persistence.

Request an AI summary and compare it with the source profile.

Show the relevant API route or test result.

LIVE DEMO

Open Candidate Triage Tool

http://127.0.0.1:5000 — works only while the app is running on your computer.